HARBOUR.SPACE
INSTITUTE OF TECHNOLOGY

CS403BKK - Software Engineering (6 ECTS)
CS404aBKK - Software Engineering, Part 2 (8 ECTS)
Syllabus, Maksym Prokopov

1| Dates of module

28 Sept 2026 - 16 Oct 2026

2| Abstract

There is a big difference between coding and engineering. This course teaches software engineering as the design and operation of a software factory: the socio-technical system that turns an idea into a safe, valuable, maintainable product and then learns from production. We cover the full lifecycle — discovery, requirements, architecture, implementation, verification, delivery, operation, incident response, maintenance, and retirement — and treat each stage as producing evidence the next stage depends on. Coding agents have made implementation cheaper and shifted scarce work toward specification, architecture, verification, risk management, and organisational learning. Agentic development is therefore used as a pressure test of engineering practice rather than as a separate topic. Students inherit a provided Python product: a FastAPI service using SQLAlchemy, Alembic, PostgreSQL, pytest, and OpenTelemetry, with a thin prebuilt web client outside the main assessed scope, already wired to CI and deployment. This mirrors professional reality and asks the question the course actually cares about: can another engineer safely change and operate this?

These sessions are taught under two course codes. Harbour.Space students enrolled under CS403BKK take this course as a 6 ECTS module and are assessed under the Section 7 “Evaluation and Grading”.

St. Gallen exchange students, enrolled under CS404aBKK, take these sessions as Part 2 of an 8 ECTS course and are assessed under the scheme set out in the CS404aBKK syllabus.

3| Learning Highlights

The course will provide an opportunity for participants to:

- Explain the full software delivery cycle: discovery, shaping, design, implementation, verification, deployment, operation, learning, and retirement.

- Turn an ambiguous product request into testable requirements, quality attributes, and an incremental delivery plan using Domain Storytelling, scenarios, and user stories.
- Identify domain boundaries and data ownership using Bounded Contexts, and apply appropriate Domain-Driven Design patterns in the server-side domain model.
- Design a service and its failure modes: APIs, external integrations, timeouts, retries, idempotency, error contracts, and threat modelling.
- Work effectively inside an existing codebase, using version control, code review, and CI/CD as collaboration and quality systems rather than as file storage and automation.
- Apply testing across unit, integration, contract, and end-to-end levels, and understand the limits of coverage.
- Release changes safely using feature flags, progressive delivery, rollback, and blast-radius control.
- Use production signals — metrics, logs, traces, alerts, and user feedback — to guide engineering decisions, and respond to a live incident under time pressure.
- Use coding agents within clear boundaries while retaining human ownership of design, verification, security, and production risk.
- Evaluate trade-offs in maintainability, cost, reliability, security, delivery speed, and user value, and sustain them as a team through documentation and decision records.

4| Required Skills or Courses

Students must be ready to modify a small web application. Before the module, they should be able to use Git branches and pull requests, trace a basic HTTP request, work with relational data, write and debug automated tests, and diagnose failures from logs. Working familiarity with Python is required, including modules, functions, classes, exceptions, packages, and type hints. FastAPI, SQLAlchemy, Alembic, and pytest are practical vehicles, not subjects taught from first principles. The module does not reteach programming, Git, HTTP, databases, or testing. The readiness package in Methodology is mandatory before Session 1.

5| Acquired Skills

- Software engineering
- Software architecture
- Domain-Driven Design
- Requirements engineering
- Software testing and quality assurance
- Continuous integration and delivery
- Observability and operations
- Incident response and resilience
- Software maintainability and system evolution
- Agile software development
- Working in a team
- Agent-assisted software development

6| Methodology

The course comprises 6 ECTS, or 180 hours of total learner workload: 45 scheduled contact hours across 15 three-hour sessions, including examinations; 15 hours of consultation, project review, and individual defence; and 120 hours of independent study and team project work. ECTS measures total workload, not classroom time. Independent work is distributed before, during, and after the teaching block; the major team submission falls in the lecture-free assessment period.

Each session combines a lecture, a supervised studio, and an evidence review. Session 8 is reserved for project work, and short quizzes run weekly.

Before Session 1, every student completes a short, ungraded readiness package in the provided Python repository: run the system, follow a guided codebase tour, fix a small defect, add or adjust one pytest test, and submit a pull request. It is a diagnostic and support mechanism, not a selection exercise. Students who cannot complete it should contact the course team before the module.

Teams inherit a running FastAPI service using SQLAlchemy, Alembic, PostgreSQL, pytest, and OpenTelemetry, plus a thin prebuilt web client they rarely modify; the system is already wired to CI and deployment. The assessed core is one Python service change taken through a minimum viable lifecycle: brief and acceptance criteria; one architecture decision record; a working vertical slice with any required API contract change; automated verification and justified CI gates; an executed release and rollback record; and a short Engineering Evidence Log covering production observation, agent use, verification, and confidence. Supporting evidence is capped at roughly one page per item. Tactical DDD, threat modelling, the live incident, deprecation analysis, and team factory design are practised in studios, quizzes, and the defence, but do not each require a separate project deliverable. Session 11 contains an injected incident; Session 12 analyses the deprecation candidate. Studio attendance is essential.

7| Evaluation and grading

decentralised – Individual Assignments and Weekly Quizzes, Digital, Individual Work, Individual Grade (40%)
Examination time: Lecture and lecture-free period.

decentralised – Group Project, Digital, Group Work, Individual Grade (35%)
Examination time: Lecture-free period.

To pass the group project, a team must demonstrate the complete minimum viable lifecycle, even for a small change. Higher grades reward better decisions, evidence, and operational safety — not more features. Sessions 4, 8, 10, and 14 are formative checkpoints; final submission is in the lecture-free assessment period.

decentralised – Individual Design and Operational Defence, oral, Individual Work, Individual Grade (15%)
Examination time: Lecture period.

central – Written Examination, analog open-ended questions plus a digital practical part, Individual Work, Individual Grade (10%, 180 min)
Examination time: Lecture period.

8| Outline

Session 1	Session 2
Coding is not software engineering. The software factory before and after coding agents. Value, risk, and bottlenecks. Tour of the provided codebase.	Product discovery and problem framing. Domain storytelling, scenarios, user stories, acceptance criteria. Requirements versus quality attributes.

Session 3

Architecture, domain boundaries, and failure-aware design. Domain-Driven Design, context maps, and architecture decision records.
Integrations, timeouts, idempotency, error contracts, and threat modelling.

Session 4

Planning an agent-assisted change. Product review, program design, delegation boundaries, and human ownership.

Session 5

Implementation discipline. Python, FastAPI, SQLAlchemy, and Alembic. Git as a collaboration and recovery system. Vertical slices.

Session 6

Tests as engineering feedback. pytest; unit, integration, contract, and end-to-end testing; Testcontainers. The limits of coverage.

Session 7

Code review, continuous integration and delivery, and the maintainability gap.

Session 8

Project work. Unstructured team build time with the professor available for consultation and mid-course review.

Session 9

Releasing safely. Deployment strategies, feature flags, progressive delivery, rollback, and blast radius.

Session 10

Observability and production feedback. Metrics, logs, traces, SLOs, dashboards, and alert quality.

Session 11

Incidents, security, and resilience. Injected live incident and graded response. Postmortems, OWASP, dependency risk, and secrets.

Session 12

Maintaining and simplifying systems. Legacy code, technical debt, refactoring, migration, deprecation, and deletion.

Session 13

Team factory design. Documentation, knowledge sharing, decision records, ownership, meeting culture, and bottleneck metrics.

Session 14

Capstone factory review. Teams present lifecycle evidence; peer review on risk and future change.

Session 15

Final exam. Scenario-based assessment and course retrospective.

9| Bibliography

- A Philosophy of Software Design, by John Ousterhout
- Software Engineering at Google: Lessons Learned from Programming Over Time, by Titus Winters, Tom Manshreck, and Hyrum Wright, O'Reilly Media, 2020
- Site Reliability Engineering, selected chapters (website)
- Designing Data-Intensive Applications, by Martin Kleppmann, selected chapters, O'Reilly Media, 2017
- Domain Storytelling: A Collaborative, Visual, and Agile Way to Build Domain-Driven Software, by Henning Schwentner and Stefan Hofer, Addison-Wesley, 2021
- Domain-Driven Design: Tackling Complexity in the Heart of Software, by Eric Evans, selected chapters, Addison-Wesley, 2003
- Modern Software Engineering, by David Farley, Addison-Wesley, 2021
- Why Software Factories Fail (or: the harness is not enough), Part I, by Dex Horthy, 2026
- Why Software Factories Fail: Turning the Lights Back On, Part II, by Dex Horthy, 2026
- Python (website), FastAPI (website), SQLAlchemy (website), Alembic (website), pytest (website)

APPENDIX| Mini CV



Maksym Prokopov
Senior Site Reliability Manager at Billie.io

Maksym is an experienced tech lead developer and Staff Site Reliability Engineer spent many years creating solutions from scratch to the production. Possesses the ability to explain complex concepts in a simple way.

He believes everyone with passion eventually becomes a professional.

Faculty webpage: https://harbour.space/faculty/maksym-prokopov